Multilevel Models II

Math 243

Michael Pearce

Outline

Last time, we...

- Introduced Multilevel Models (MLMs) as a method of accounting for "clustering" in data
- Discussed two kinds of MLMs: Random Intercepts and Random Slopes

In today's class, we will...

- Implement MLMs in R

Section 1

Implementation in R

Grades within Schools

Let's simulate data according to a true "random intercepts" model:

- 50 schools (groups), 10 students per school (observations)
- Intercept varies by school (random intercepts)
- Relationship between hours studied (X) and grade (Y) same across schools

Grades within Schools

Let's simulate data according to a true "random intercepts" model:

- 50 schools (groups), 10 students per school (observations)
- Intercept varies by school (random intercepts)
- Relationship between hours studied (X) and grade (Y) same across schools

The true data will take the form:

$$Grade_{g[i]} = (\beta_0 + \gamma_g) + \beta_1 Hours_{g[i]} + \eta_{g[i]}$$
$$\gamma_g \sim N(0, \sigma^2_\gamma)$$
$$\eta_{g[i]} \sim N(0, \sigma^2_\eta)$$

with $\beta_0 = 70$, $\beta_1 = 1$

lmer function: Random Intercepts

We will use the `lmer` function in the `lmerTest` package to fit MLMs.

```
library(lmerTest)
lmer(grade ~ hours + (1|school), data = school_data)
```

`lmer` is very similar to `lm`:

- Include a model formula (e.g., `y ~ x`) and data.
- (Fixed effect) intercept is automatically included.
- `(1|school)` adds in random intercepts *by school*
- `hours` on its own adds the *fixed effect* for `hours` (i.e., not varying by school)

`lmer` function: Random Slopes

Suppose I wanted to add **random slopes**, i.e., slopes varying by `school`

```
library(lmerTest)
lmer(grade ~ hours + (hours|school), data = school_data)
```

- `(hours | school)` adds in random slopes **and** random intercepts *by school*
- `hours` still needs to be included **on its own**, otherwise there's no "overall" slope term, β_1

Examining lmer output

```
model_ri <- lmer(grade ~ hours + (1|school), data = school_data)
summary(model_ri)
```

```
## Linear mixed model fit by REML. t-tests use Satterthwaite's method [
## lmerModLmerTest]
## Formula: grade ~ hours + (1 | school)
##    Data: school_data
##
## REML criterion at convergence: 1756
##
## Scaled residuals:
##     Min      1Q  Median      3Q     Max
## -2.4681 -0.6088 -0.0109  0.7218  3.3508
##
## Random effects:
##  Groups   Name        Variance Std.Dev.
##  school   (Intercept) 17.15    4.141
##  Residual              1.18    1.086
## Number of obs: 500, groups:  school, 50
##
## Fixed effects:
##             Estimate Std. Error        df t value Pr(>|t|)
## (Intercept) 7.041e+01  5.945e-01 5.133e+01   118.4   <2e-16 ***
## hours       1.004e+00  9.048e-03 4.498e+02   111.0   <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Correlation of Fixed Effects:
##       (Intr)
## hours -0.152
```

Displays estimates of variances parameters of "random effects", i.e., group-varying components.

- Not super interpretable!

Displays estimates of "fixed effects", i.e., overall intercepts/slopes.

- p-values just like in regular linear regression: Two-sided test of

$$H_0 : \beta_j = 0 \text{ vs. } H_A : \beta_j \neq 0$$

Examining lmer output

We can examine estimated coefficients by group using the coefficients function

```
coefficients(model_ri)$school
```

```
##           (Intercept)    hours
## School1      67.30016 1.004184
## School10     68.85067 1.004184
## School11     77.60559 1.004184
## School12     72.04892 1.004184
## School13     66.61849 1.004184
## School14     58.78437 1.004184
## School15     75.10193 1.004184
## School16     69.19779 1.004184
## School17     70.51404 1.004184
## School18     74.63540 1.004184
## School19     74.03204 1.004184
## School2      70.84321 1.004184
## School20     73.09673 1.004184
## School21     74.49047 1.004184
## School22     73.47234 1.004184
## School23     70.21767 1.004184
```

Inference with MLMs

For a more succinct view of p-values, we can use `summary([MODEL NAME])$coef`

```
summary(model_ri)$coef
```

```
##                Estimate  Std. Error        df  t value    Pr(>|t|)
## (Intercept) 70.414226 0.594542055  51.33425 118.4344 2.92062e-64
## hours        1.004184 0.009047892 449.81124 110.9854 0.00000e+00
```

For confidence intervals, we use `confint()`

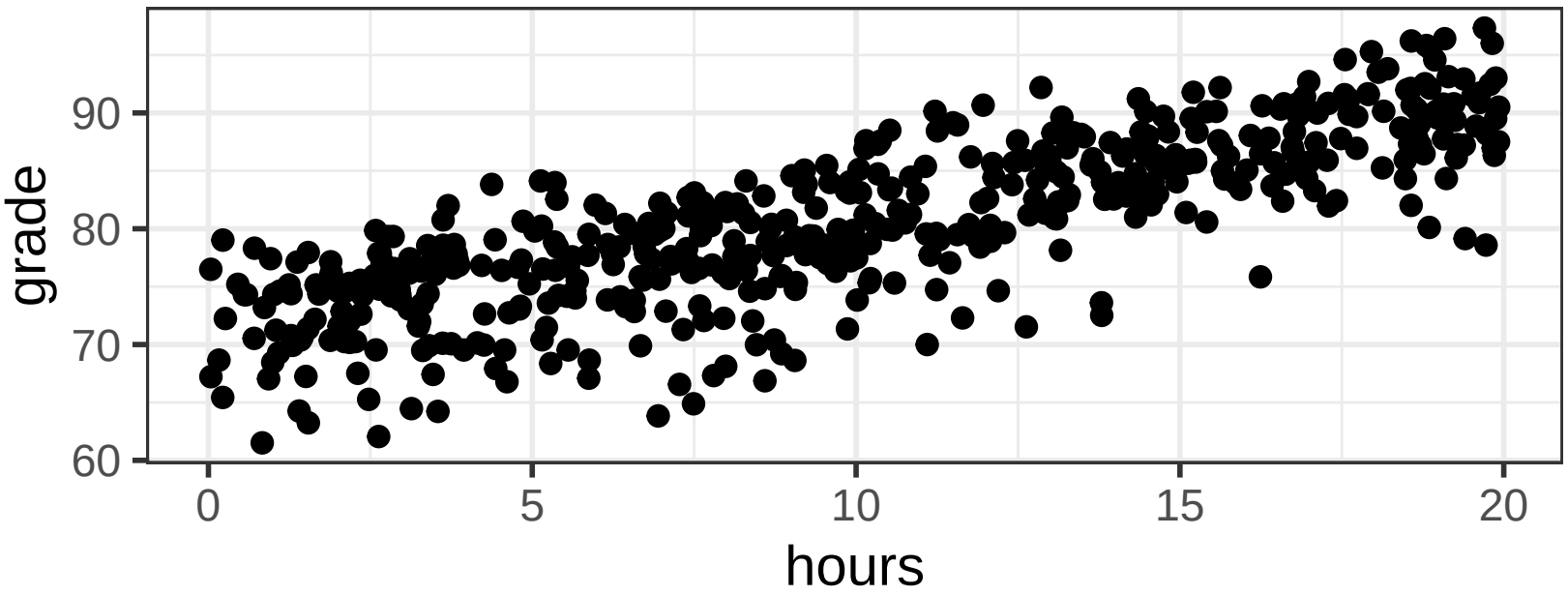
```
confint(model_ri, level = 0.95)
```

```
##                   2.5 %    97.5 %
## .sig01        3.4049688  5.061517
## .sigma        1.0180571  1.160235
## (Intercept) 69.2389313 71.589558
## hours        0.9864332  1.021936
```

Plotting Models

Plotting is harder! We start with a standard ggplot of our data

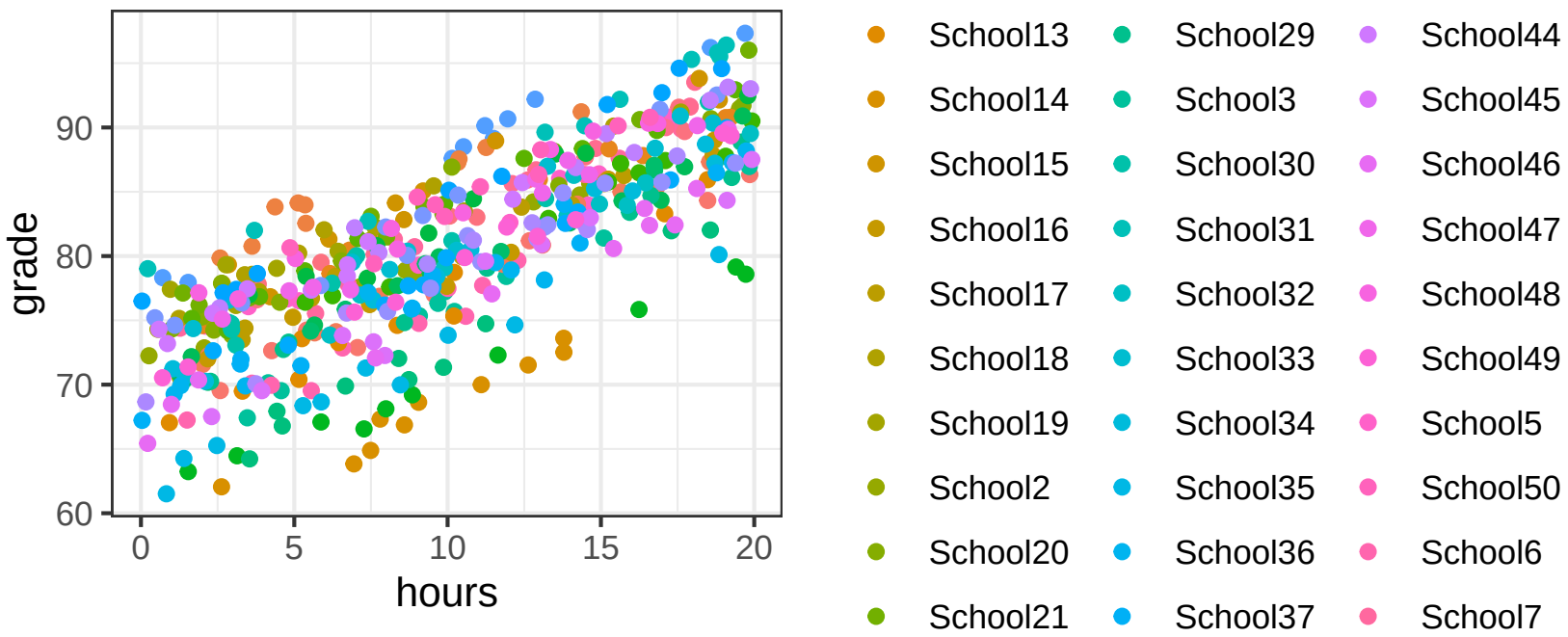
```
ggplot(school_data,aes(x=hours,y=grade))+theme_bw(base_size=15)+
  geom_point()
```



Plotting Models

Let's add color by school:

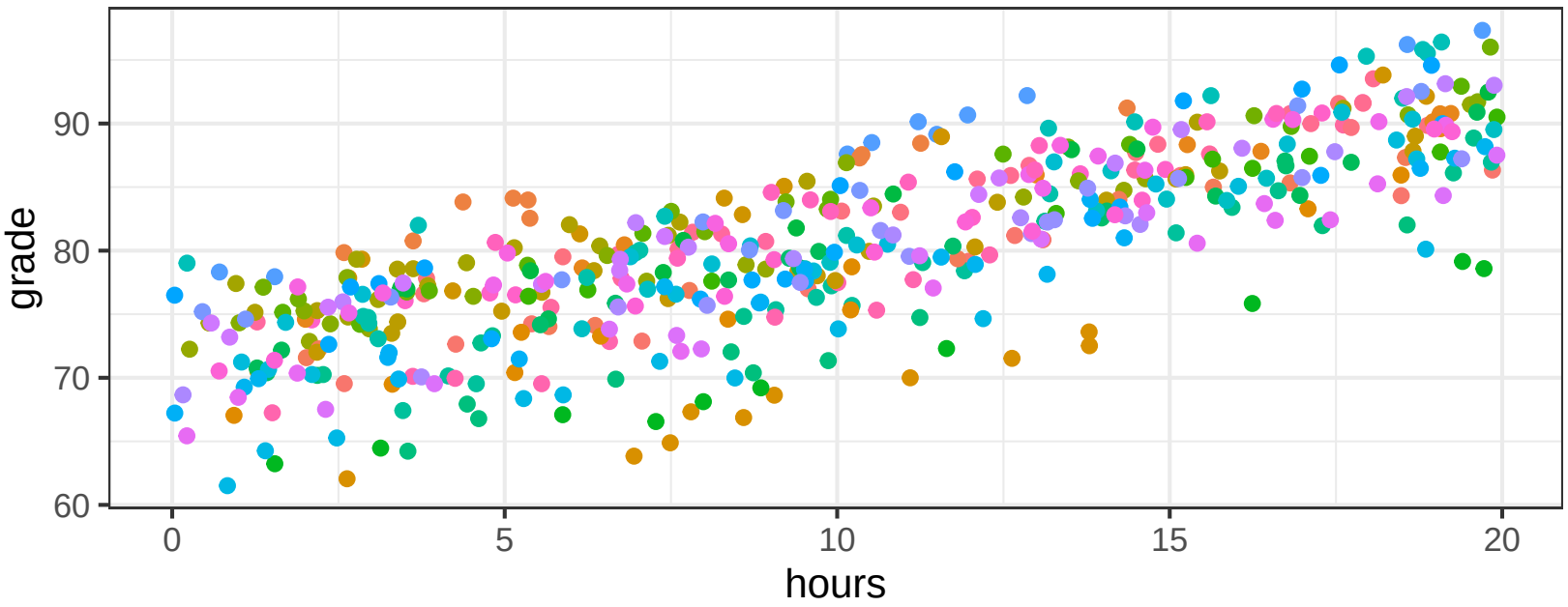
```
ggplot(school_data,aes(x=hours,y=grade,color=school))+theme_bw()+
  geom_point()
```



Plotting Models

Oof! Let's remove the legend:

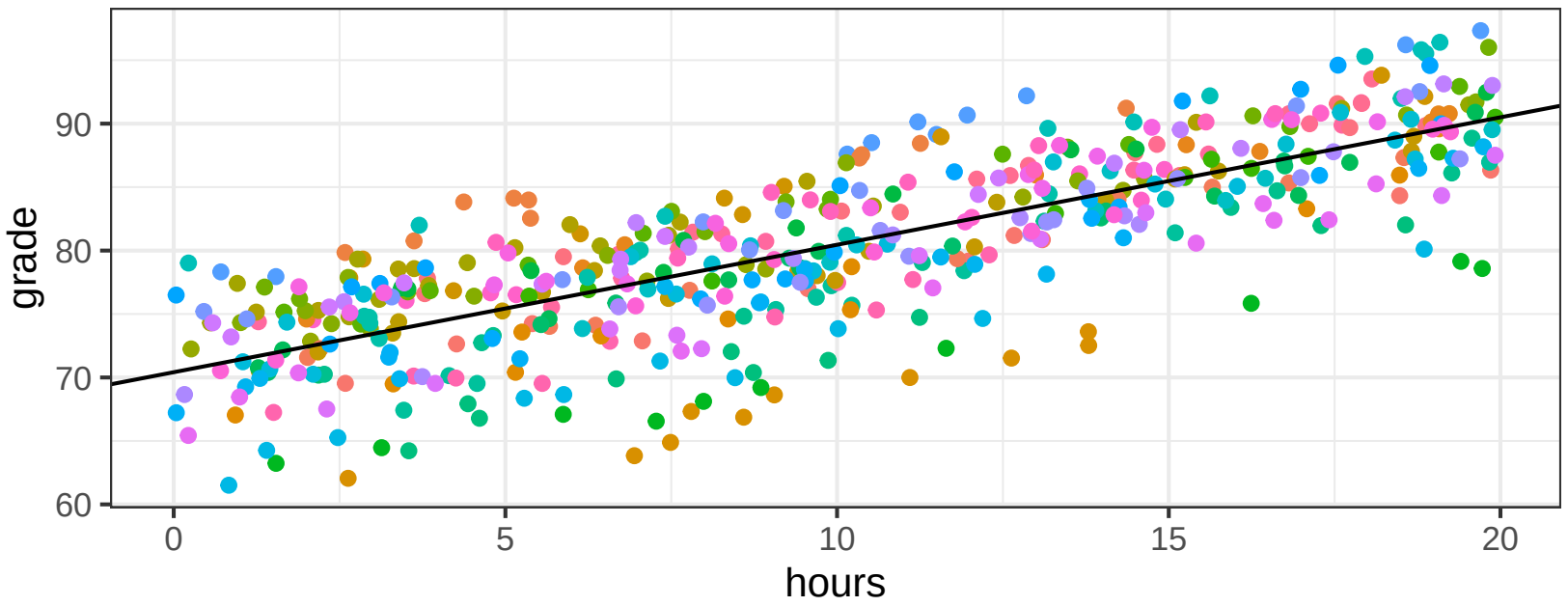
```
ggplot(school_data,aes(x=hours,y=grade,color=school))+theme_bw()+
  geom_point()+theme(legend.position = "none")
```



Plotting Models

We could add the "overall" regression line using geom_abline()

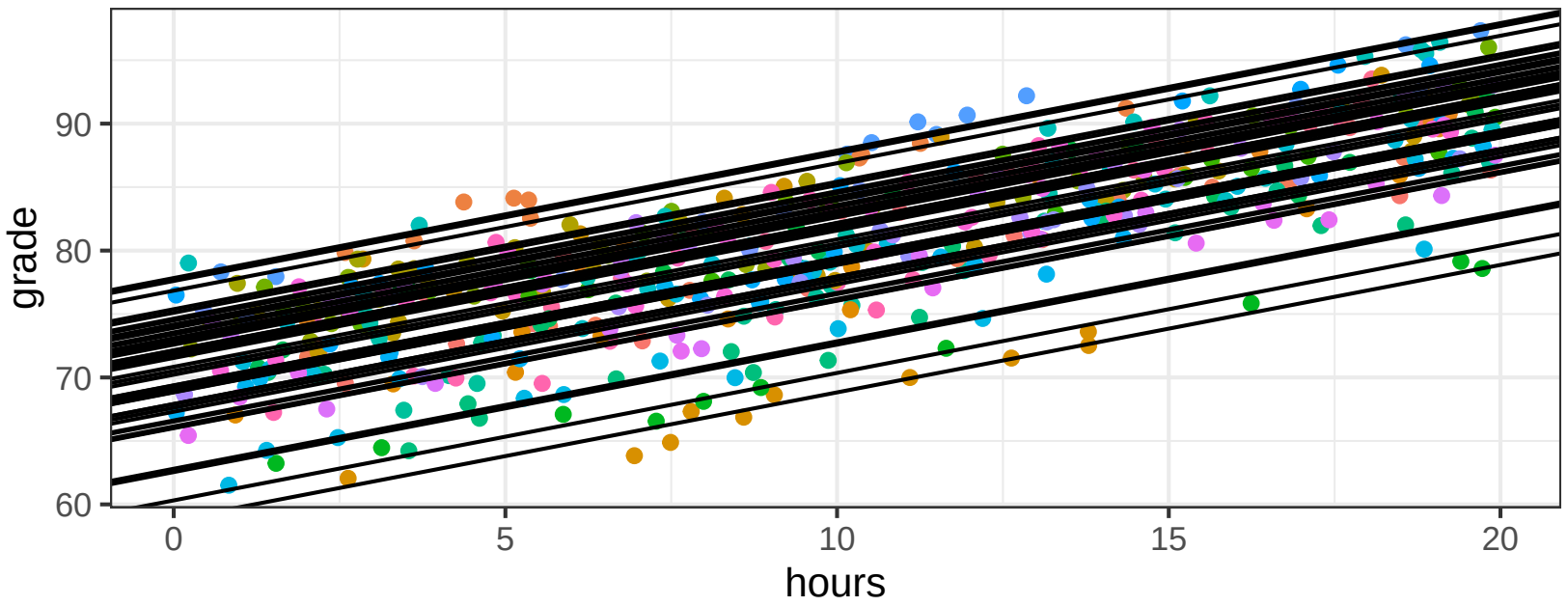
```
coefficients <- summary(model_ri)$coef[,"Estimate"]
ggplot(school_data,aes(x=hours,y=grade,color=school))+theme_bw()+
  geom_point()+theme(legend.position = "none")+
  geom_abline(intercept=coefficients[1],slope=coefficients[2])
```



Plotting Models

Or, school-specific regression lines

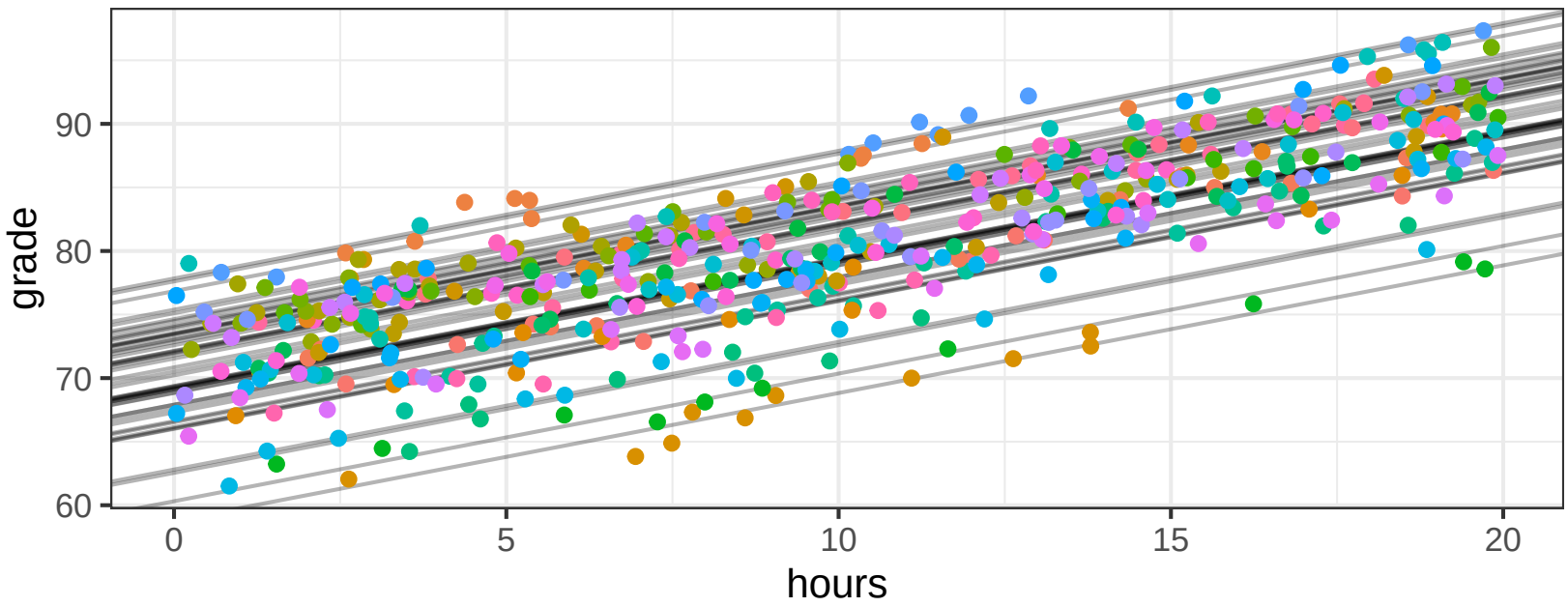
```
ggplot(school_data,aes(x=hours,y=grade,color=school))+theme_bw()+
  geom_point()+theme(legend.position = "none")+
  geom_abline(intercept=coefficients(model_ri)$school[,1],
              slope=coefficients(model_ri)$school[,2])
```



Plotting Models

Change order of layer functions; make lines semi-transparent (e.g., `alpha=0.3`)

```
ggplot(school_data,aes(x=hours,y=grade,color=school))+theme_bw()+
  geom_abline(intercept=coefficients(model_ri)$school[,1],
      slope=coefficients(model_ri)$school[,2],alpha=0.3)+
  geom_point()+theme(legend.position = "none")
```



Section 2

Recap

Recap

Today, we...

- Implemented MLMs in R and plotted them
- Rest of Today: Work on Lab!